Technical University of Crete
School of Electrical and Computer Engineering
Course: Randomized Algorithms
Exercise 4 (100/1000)
Report Delivery Date: 20/12/2024
Instructor: Athanasios P. Liavas

Paraskevopoulou Despoina
2021030135

Johnson-Lindenstrauss Lemma

In the first part of this exercise we studied the Johnson-Lindenstrauss Lemma. At first, we started by generating a random matrix $\mathbf{X} \in \mathbb{R}^{d \times m}$ using `randn()` from Matlab, where each column on $\mathbf{X}$ represents a data point in $\mathbb{R}^d$.

```
%% 1
d = 1e4; m = 50;
X = randn(d,m);
```

We, then, proceeded to compute a vector which contains the $\frac{m(m-1)}{2}$ pairwise distances between the columns of $\mathbf{X}$.

```
%% 2
t = m*(m-1)/2; index = 1;
dist = zeros(1,t);

for i=1:m
    for j=i+1:m
        dist(index) = norm(X(:,i)-X(:,j));
        index = index + 1;
    end
end
```

Given specific ϵ and δ we computed the reduced dimension n according to the J-L Lemma. From the Lemma we know that

$$\epsilon = \sqrt{\frac{6log(\frac{2|\mathbf{Q}|}{\delta})}{n}} \Rightarrow$$

$$n = \frac{6log(\frac{2|\mathbf{Q}|}{\delta})}{\epsilon^2}$$

We set the $|\mathbf{Q}| = \frac{m(m-1)}{2}$ and for $d = 10^4$ and $m = 50$ we get $n = 6063$.
For the next step, we generated the random matrix $\mathbf{W} \in \mathbb{R}^{n \times d}$. The matrix $\mathbf{W}$ according to the J-L Lemma is a random matrix such that every $W_{i,j}$ is an independent normal random variable $W_{i,j} \sim \mathcal{N}(0,1)$, so we use the `randn()` function of Matlab. Afterwards, we computed the matrix $\mathbf{Y}$, where $\mathbf{Y} = \mathbf{WX}$.

```
%% 4
n_ = floor(n);
W = randn(n_,d);
W_ = W / sqrt(n_);
Y = W_*X ;
```

We moved on to compute the $\frac{m(m-1)}{2}$ pairwise distances between the columns of matrix $\mathbf{Y}$. For all possible pairs (i, j) we plotted the quantities:

$$\sqrt{1-\epsilon}\|X(:,i) - X(:,j)\|_2 \tag{1}$$

$$\sqrt{1+\epsilon}\|X(:,i) - X(:,j)\|_2 \tag{2}$$

$$\|Y(:,i) - Y(:,j)\|_2 \tag{3}$$

```
%% 5,6
index = 1;
t2 = m*(m-1)/2;
dist_Y = zeros(1,t2);
for i=1:m
    for j=i+1:m
        dist_Y(index) = norm(Y(:,i)-Y(:,j));
        quant_1(index) = sqrt(1-epsilon)*norm(X(:,i)-X(:,j));
        quant_2(index) = sqrt(1+epsilon)*norm(X(:,i)-X(:,j));

        index = index + 1;
    end
end

scatter(1:t2, dist_Y, 40, 'r');
scatter(1:t2, quant_1, 40, 'g');
scatter(1:t2, quant_2, 40, 'b');
```

The results we got from the above are:

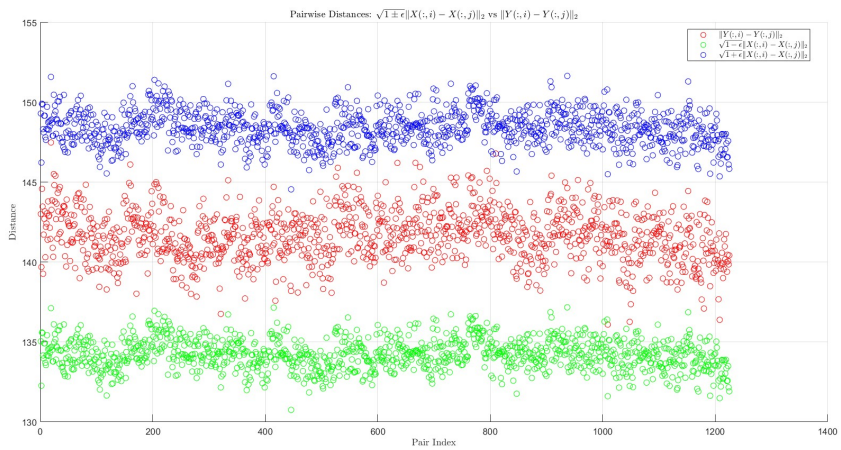

Figure 1: The three vectors for $d = 10^4$ and $m = 50$

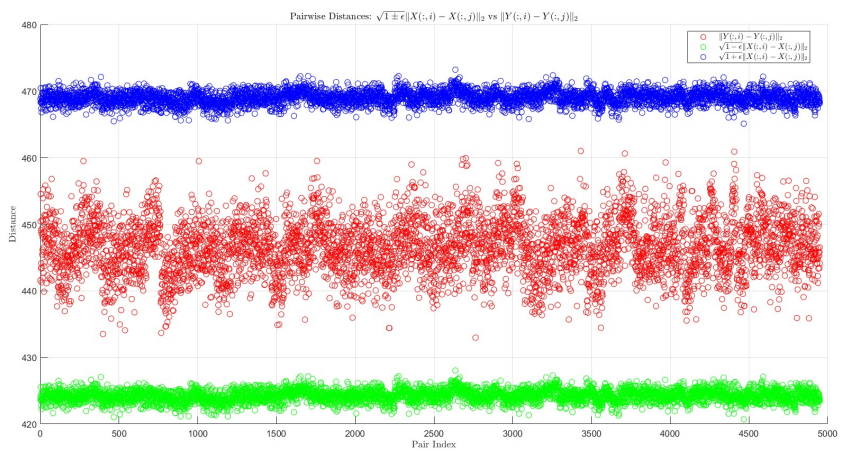

Figure 2: The three vectors for $d = 10^5$ and $m = 100$

From the plots we observe that for almost all the pairs the inequality below holds:

$$\sqrt{(1-\epsilon)}\|X(:,i) - X(:,j)\|_2 \leq \|Y(:,i) - Y(:,j)\| \leq \sqrt{(1+\epsilon)}\|X(:,i) - X(:,j)\|_2$$

Actually we count the pairs for which the above inequality holds in order to compute the empirical probability of success, which turns out to be 1.
From the theory, we know that

$$\Pr((1-\epsilon)n \leq \|\mathbf{Wx}\|^2 \leq (1+\epsilon)n) \geq 1 - 2e^{-\epsilon^2 n/6}$$

which is an equivalent inequality to the one we proposed above, so

$$\Pr(\text{success}) \geq 1 - 2e^{-\epsilon^2 n/6}$$

which in our case $1 - 2e^{-\epsilon^2 n/6} = 0.999$, for $\epsilon = 0.1$.

2-SAT Algorithm

In the second part of this exercise, we studied the randomized 2-SAT algorithm with n Boolean variables, $\mathbf{x} \in \{0,1\}^n$ and $\mathcal{K}$ 2-clauses. For the implementation we followed the steps below:

1. We generated $\mathcal{K}$ different random pairs (i_1, i_2) where $i_1 \neq i_2$ and $i_1, i_2 = 1, ..., n$. The pairs represent the boolean variables x_{i_1}, x_{i_2} that participate in the $\mathcal{K}$ clauses.

```
%% 1
clauses = zeros(K,2);

for i=1:K
    clauses(i,:) = randperm(n,2);
end
```

2. We, then generated an arbitrary truth assignment represented by $\mathbf{s} \in \{0,1\}^n$.

```
%% 2
S = zeros(1,n);
for i=1:n
    S(i) = randi([0 1]);
end
```

3. Then, we generated a $(\mathcal{K} \times 2)$ matrix that represents with 1 which of the literals participating in the clauses is in the form $\bar{s_i}$. In other words, the matrix has ones where there is a *negation*.

```
%% 3
lit_clauses = zeros (K,2);

for i = 1:K
    index = randi([1 2]);
    if(index==1)
        if(S(clauses(i,1))==0)
            lit_clauses(i,1) = 1;
        end
    end
    if(index==2)
        if(S(clauses(i,2))==0)
            lit_clauses(i,2) = 1;
        end
    end
end
```

4. In this step, we wrote a function `check_2SAT(x, clauses, lit_clauses, K)` which returns a $\mathcal{K}$-dimensional boolean vector, with 1 in the *i-th* position, if the *i-th* clause is satisfied by the truth assignment $\mathbf{x}$, and 0 otherwise.

```
function pos_sat_clause=check_2SAT(x,clauses,lit_clauses,K)
    pos_sat_clause = zeros(1,K);

    for i = 1:K
        t1 = clauses(i,1); t2 = clauses(i,2);

        if lit_clauses(i,1)==1
            if x(t1)==0
                pos_sat_clause(i) = 1;
            elseif lit_clauses(i,2)==1
                if x(t2)==0
                    pos_sat_clause(i) = 1;
                else
                    pos_sat_clause(i) = 0;
                end
            elseif lit_clauses(i,2) == 0
                if x(t2) == 1
                    pos_sat_clause(i) = 1;
                else
                    pos_sat_clause(i) = 0;
                end
            end

        elseif lit_clauses(i,1)==0
            if x(t1)==1
                pos_sat_clause(i) = 1;

            elseif x(t1)==0
                if lit_clauses(i,2) == 1
                    if x(t2) == 0
                        pos_sat_clause(i) = 1;
                    else
                        pos_sat_clause(i) = 0;
                    end
                elseif lit_clauses(i,2) == 0
                    if x(t2) == 0
                        pos_sat_clause(i) = 0;
                    else
                        pos_sat_clause(i) = 1;
                    end
                end
            end
        end
    end
end
```

5. In this step we proceeded with the implementation of the 2-SAT algorithm. From the implementation we observe that the problem is always solved. We run the algorithm for various n, $\mathcal{K}$ and m. For better understanding we plotted the number of iterations for various values of n ($\mathcal{K} = n$), and we got the following results:

```
Truth assignment found for n = 100 after 28 iterations!
Truth assignment found for n = 200 after 70 iterations!
Truth assignment found for n = 500 after 129 iterations!
Truth assignment found for n = 1000 after 333 iterations!
Truth assignment found for n = 2000 after 556 iterations!
Truth assignment found for n = 4000 after 1157 iterations!
Truth assignment found for n = 10000 after 2745 iterations!
Truth assignment found for n = 20000 after 5918 iterations!
Truth assignment found for n = 40000 after 11661 iterations!
Truth assignment found for n = 100000 after 29715 iterations!
```

Figure 3: Iterations with respect to n

We observe that as the n grows so do the iterations, with an average ratio $\frac{n}{iterations} = 3.44$ which is expected concerning the complexity of the algorithm, We also, plotted the number of iterations with respect to n and we got seeing a linear relation between these variables.

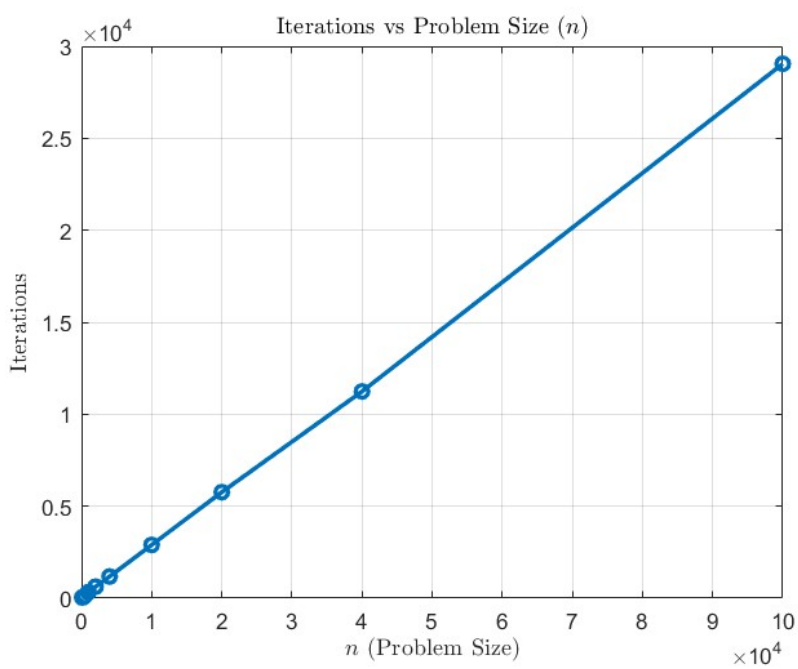

Figure 4: Plot of iterations with respect to n

3-SAT

For this part of the exercise we implemented the 3-SAT algorithm. We know that the 3-SAT problem is an NP-hard problem so we do not expect to have a polynomial time algorithm for it. We made the following modifications to the 2-SAT code:

- First, now the clauses matrix is a $\mathcal{K} \times 3$ matrix, as is the literal clauses matrix.
- We modified the `check_2SAT` function, so it is simpler and easier to implement in the 3-SAT problem.

We run the algorithm for various values of n, $n = [50, 100, 150, 200, 250, 300, 325, 350]$, $\mathcal{K} = 5n$ and $m = 10$.

```
Truth assignment found for n = 50 , K = 250 after 45 iterations.
Iterations for finding truth assignment: 45
Truth assignment found for n = 100 , K = 500 after 197 iterations.
Iterations for finding truth assignment: 197
Truth assignment found for n = 200 , K = 1000 after 711 iterations.
Iterations for finding truth assignment: 711
Truth assignment found for n = 500 , K = 2500 after 7976 iterations.
Iterations for finding truth assignment: 7976
```

Figure 5: Iterations for various n and K

From this run, we can observe the exponential growth of the number of iterations. In order to get a better feeling, we plotted the number of iterations versus n values.

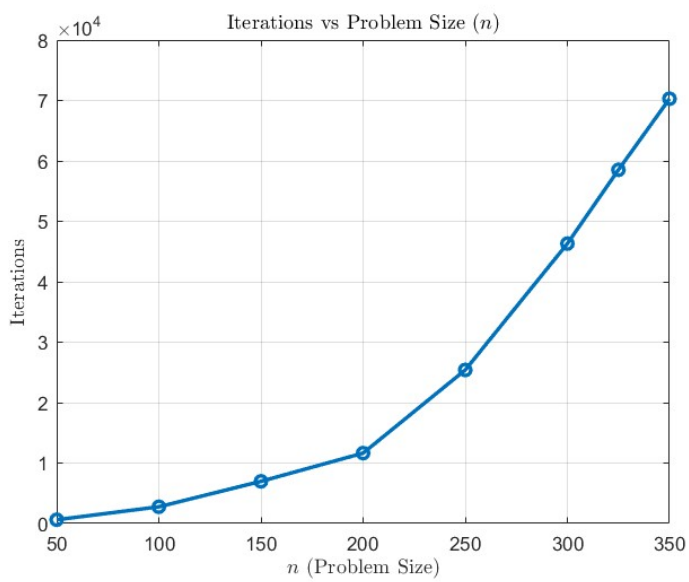

Figure 6: Iterations versus various values of n

We see that after 300 the number of iterations rockets proving our statement from above. Comparing this output with the output for 2-SAT we clearly see their differences in run time.